



Faculty of Computing and IT
Bachelor of Science in Applied Information Technology

CIT211 – Mobile Software Development
Final Assignment

Coursework Specification:
Design and Development of a Fashion Store Mobile
Application

Name: M.R.F Rukaiya
Student ID: 23da2-0912

Introduction

LuxeWear is a Flutter-based mobile shopping application that implements a polished e-commerce UI kit with authentication, product browsing, cart management, checkout flow, and profile features. The app is designed for cross-platform use and integrates Firebase services for authentication and cloud data storage.

System Overview

- Platform: Flutter mobile app
- Backend services: Firebase Authentication and Cloud Firestore
- Core modules:
 - Authentication: login, registration, Google sign-in
 - Product browsing: home screen, product listing, product detail
 - Cart and checkout: add to cart, quantity management, checkout navigation
 - User profile: settings, order history, payment methods, support
Favorites and notifications

UI/UX Design

- Clean, luxury-themed palette based on green, white, and dark accents
- Material 3 design with rounded cards, elevated buttons, and soft shadows
- Google Fonts used for typography
- Mobile-first layout with:
 - searchable home and product listings
 - horizontal category chips
 - responsive product grid
 - large hero product imagery
 - sticky checkout button in product detail
- User-friendly auth screens with validation and progress indicators
- Navigation using named routes and bottom navigation bar for primary sections

System Architecture

- Entry point: `lib/main.dart`
- App bootstrap:
 - `Bootstrapper` initializes Firebase asynchronously
 - `MyApp` configures theme, routes, and providers
- Providers:
 - `AppAuthProvider` handles sign-in, registration, Google sign-in, and sign-out
 - `CartProvider` syncs cart items from Firestore and updates cart state
 - `FavoritesProvider` syncs favorite product IDs from Firestore
- Data layer:
 - `FirestoreService` encapsulates Firestore access and business logic
 - `Product` model defines product schema and serialization
- Navigation architecture:
 - Named routes from authentication to main screens
 - `MainAppScreen` uses `BottomNavigationBar` for Home, Shop, Cart, Profile

Database Design

- Firestore collections and document structure inferred from code:
 - **`products`** - stores product documents with:
 - **`name`**, **`price`**, **`imageUrl`**, **`category`**, **`colors`**, **`sizes`**, **`description`**
 - **`favorites`** - favorite product documents keyed by product ID
 - **`cart`** - cart item documents keyed by **`productId\_size`**
 - **fields:** **`productId`**, **`size`**, **`quantity`**
 - **`addresses`** - address documents
 - **`cards`** - payment card documents
 - **`orders`** - orders with total, shipping address, payment data, items list
 - **`notifications`** - user notifications
- Product model supports flexible category, color, and size data
- FirestoreService streams data to UI components for real-time updates

Implementation

- **Authentication:**
 - Email/password login and registration
 - Google sign-in via **`google\_sign\_in`**
 - Validation for email format and password length
- **Product browsing:**
 - **`HomeScreen`** with categories, featured hero section, and search bar
 - **`ProductListingScreen`** with real-time Firestore product stream, category filter, search, and favorites toggling
 - **`ProductDetailsScreen`** with size selection, product details expansion, and checkout action
- **Cart management:**
 - Add to cart, update quantity, remove items, clear cart
 - Cart state synchronized with Firestore using streams
- **Favorites:**
 - Toggle favorite state stored under user favorites collection
 - Favorite state persisted per authenticated user
- **Profile and settings:**
 - Profile-related route names and screens exist for settings, privacy, support, payment, and orders

Testing and Results

- Project includes **`flutter\_test`** as a dev dependency in **`pubspec.yaml`**.
- No explicit test files were found in the current workspace.
- Application is designed for runtime validation through UI flows and Firestore streams.
- Key runtime behaviors:
 - Firebase initialization guard with **`FutureBuilder`**
 - Async loading indicators for auth and data operations
 - Snackbars for validation and error messages

Challenges and Solutions

- Firebase initialization was handled using `WidgetsFlutterBinding.ensureInitialized` and `Firebase.initializeApp` inside `Bootstrapper``.
- Asynchronous user state attachment was managed inside `MainAppScreen` using authenticated user checks and provider attachment.
- Real-time synchronization of cart and favorites required Firestore stream listeners and provider notification.
- Product images and network loads include fallback UI with placeholders and error builders.
- User interactions were enhanced with immediate feedback using `Snackbars` and loading spinners.

Conclusion and Future Improvements

- Conclusion:
 - The project presents a strong Flutter e-commerce foundation with Firebase integration, clean UI, and modular architecture.
 - It covers essential shopping app flows, including authentication, catalog browsing, favorites, cart management, and checkout navigation.
- Future improvements:
 - Add automated unit tests and widget/integration tests
 - Implement full checkout payment processing and card validation
 - Add offline caching and network error handling
 - Expand order history details and profile personalization
 - Improve search functionality with backend indexing and better filter logic
 - Add accessibility improvements and localization support
 - Refine responsive layout for tablets and larger screens

References

YouTube Videos

Ai Agents (Claude, Open Ai)